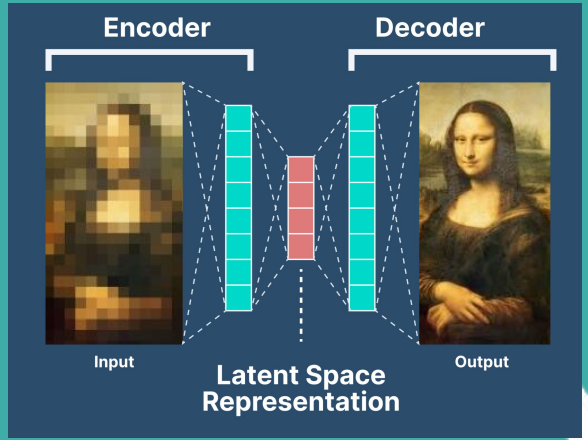
Deep Learning
CSE641/ECE555

Lecture 6 | Take your own notes during lectures

Vinayak Abrol <abrol@iiitd.ac.in>



INDRAPRASTHA INSTITUTE *of* INFORMATION TECHNOLOGY **DELHI**

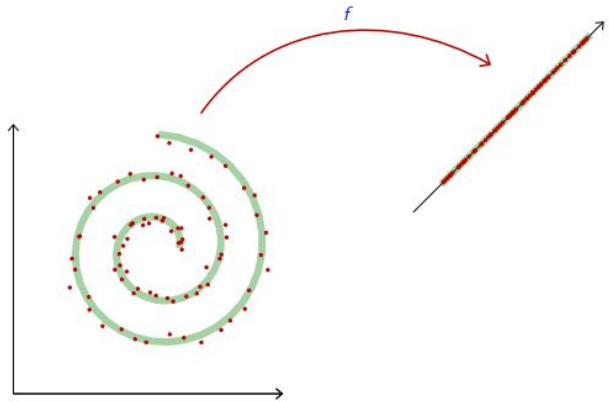
AutoEncoders (AE)

- Thinking beyond classification and regression, often our model is finding "meaningful features or degrees of freedom" that describe the variations in signal, and that too in less number of dimensions (as we go deeper).

- DNNs are non-linear (many to one mappings) hence not invertible directly.
- Hence, we need a way to learn an inverse mapping to recover our input signal.
- And this process should work for a variety of inputs (even if data is unlabeled).

This motivates the use of deep architectures for Signal Synthesis

The idea of Signal Synthesis can be extended to Generative modelling (synthesise similar but new signals).

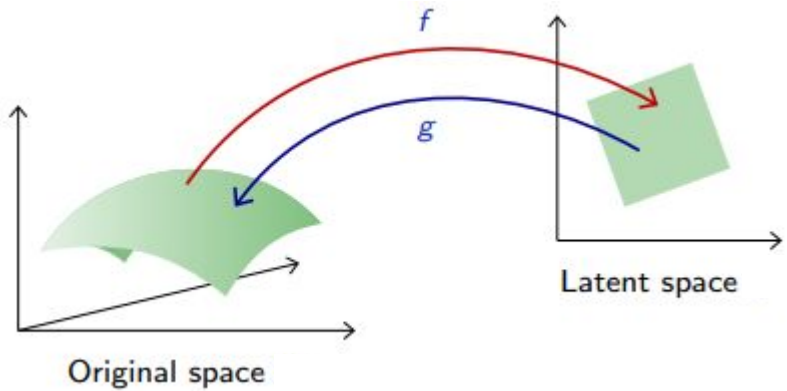


AutoEncoders (AE)

- An autoencoder maps a space to itself and is [close to] the identity on the data.
- If Latent Space is low dimensional we achieve dimensionality reduction.
  - This is not necessary for every use case but preferable.

- Mathematically

$$\mathbb{E}_{X\sim q}[\|X - g \circ f(X)\|^2] = 0$$



$$w_f, w_g = \arg\min \frac{1}{N} \sum_{n=1}^{N} \|x_n - g(f(x_n, w_f), w_g)\|^2$$

if f and g are linear; optimal solution is PCA

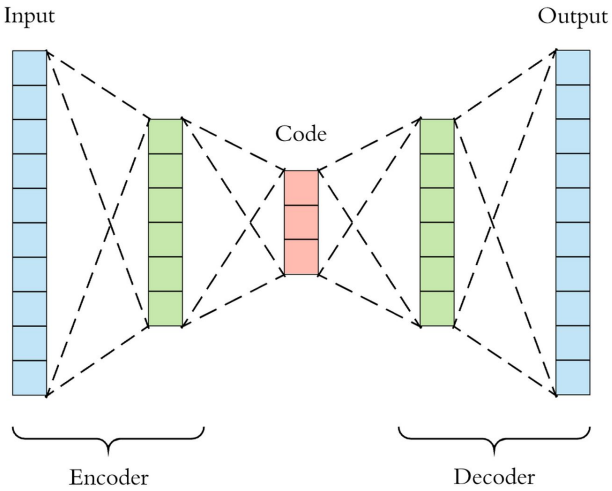
AutoEncoders (AE)

- An autoencoder maps a space to itself and is [close to] the identity on the data.
- If Latent Space is low dimensional we achieve dimensionality reduction.
  - This is not necessary for every use case but preferable.

- Mathematically

$$\mathbb{E}_{X\sim q}[\|X - g \circ f(X)\|^2] = 0$$

$$X - W_g W_f X = 0 \text{ when } W_g W_f = I$$
$$\implies W_g = W_f^{-1}$$
$$\text{and } W_g = W_f^T \ ?$$



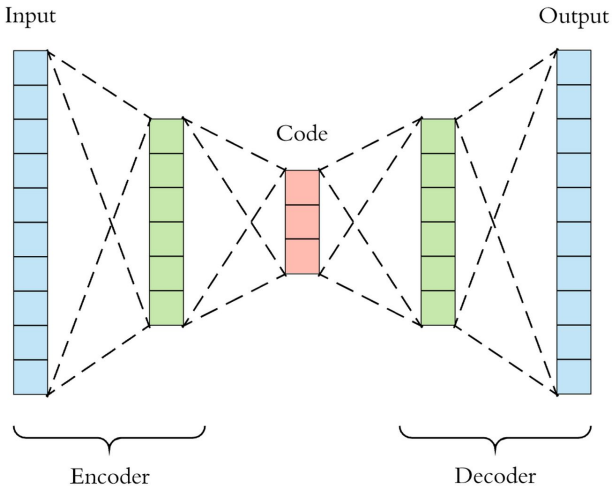
if f and g are linear; optimal solution is PCA

AutoEncoders (AE)

Autoencoders consist of 3 parts:

- Encoder: A module that compresses the input data into an encoded representation that is typically several orders of magnitude smaller than the input data.
- Bottleneck/Latent: A module that contains the compressed knowledge representations and is therefore the most important part of the network.
- Decoder: A module that helps the network “decompress” the knowledge representations and reconstructs the data back from its encoded form.

  The output is then compared with a ground truth.



The smaller the bottleneck, the lower the risk of overfitting. However, very small bottlenecks would restrict the amount of information thus increases the chances of important information slipping out.

AutoEncoders (AE)

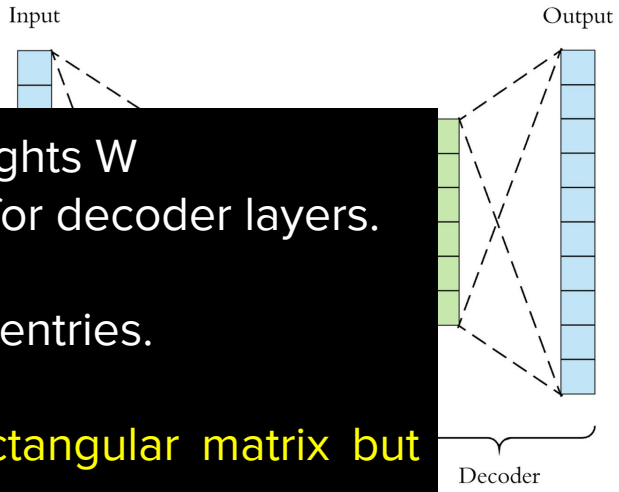
Autoencoders consist of 3 parts:

- Encoder: A module that compresses the input data into an enc
orders

- Bottlen
compre
therefo

- Decode
“decom
reconstructs the data back from its encoded form.

The output is then compared with a ground truth.



So in a nutshell for each encoder layer weights W
we need some equivalent weights $W^{-1} \backsimeq W^{T}$ for decoder layers.

Note: focus on shape not the actual matrix entries.

MLP operation is straightforward with rectangular matrix but what about CNN?

The smaller the bottleneck, the lower the risk of overfitting. However, very small bottlenecks would restrict the amount of information thus increases the chances of important information slipping out.

TansposeConvolution

Think:

- In forward pass a input signal is mapped from high dimensions to a lower dimensional vector.
- In backward pass error/gradient signal is mapped from low dimensions to high.
- Idea: Mimic the operation as in backward pass to construct a layer to increase signal dimension from latent vector.

$$y_i = (x * \kappa)_i = \sum_a x_{i+a-1}\kappa_a = \sum_u x_u \kappa_{u-i+1}$$

$$\partial_{x_u}\mathcal{L} = \sum_i \partial_{y_i}\mathcal{L}\ \partial_{x_u} y_i = \sum_i \partial_{y_i}\mathcal{L}\ \kappa_{u-i+1}$$

TansposeConvolution

Observe:

- In the first case, the filter and the signal are visited in the same order, as indexed by **'u'**
- In the second case, the derivative and the filter are visited in opposite directions, as indexes by **'i'**. The filter is "flipped" hence called Transpose

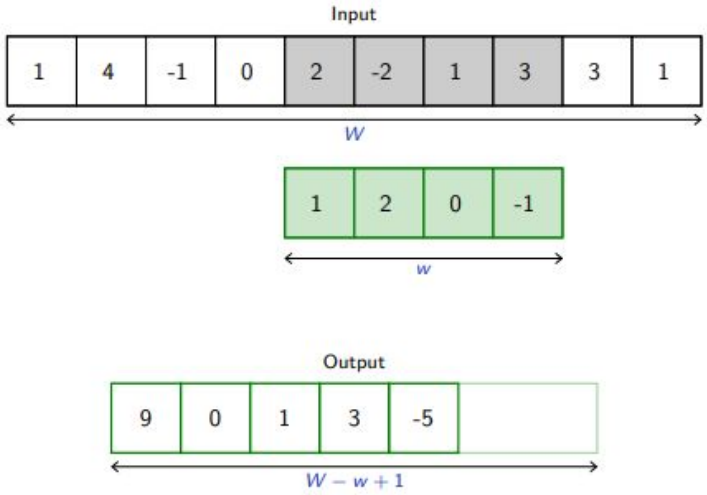
$$y_i = (x * \kappa)_i = \sum_a x_{i+a-1}\kappa_a = \sum_u x_u \kappa_{u-i+1}$$

Remember $W_g = W_f^T$

$$\partial_{x_u}\mathcal{L} = \sum_i \partial_{y_i}\mathcal{L}\ \partial_{x_u} y_i = \sum_i \partial_{y_i}\mathcal{L}\ \kappa_{u-i+1}$$

$$\begin{pmatrix} \kappa_1 & \kappa_2 & \kappa_3 & 0 & 0 & 0 & 0 \\ 0 & \kappa_1 & \kappa_2 & \kappa_3 & 0 & 0 & 0 \\ 0 & 0 & \kappa_1 & \kappa_2 & \kappa_3 & 0 & 0 \\ 0 & 0 & 0 & \kappa_1 & \kappa_2 & \kappa_3 & 0 \\ 0 & 0 & 0 & 0 & \kappa_1 & \kappa_2 & \kappa_3 \end{pmatrix}^T = \begin{pmatrix} \kappa_1 & 0 & 0 & 0 & 0 \\ \kappa_2 & \kappa_1 & 0 & 0 & 0 \\ \kappa_3 & \kappa_2 & \kappa_1 & 0 & 0 \\ 0 & \kappa_3 & \kappa_2 & \kappa_1 & 0 \\ 0 & 0 & \kappa_3 & \kappa_2 & \kappa_1 \\ 0 & 0 & 0 & \kappa_3 & \kappa_2 \\ 0 & 0 & 0 & 0 & \kappa_3 \end{pmatrix}$$

TansposeConvolution

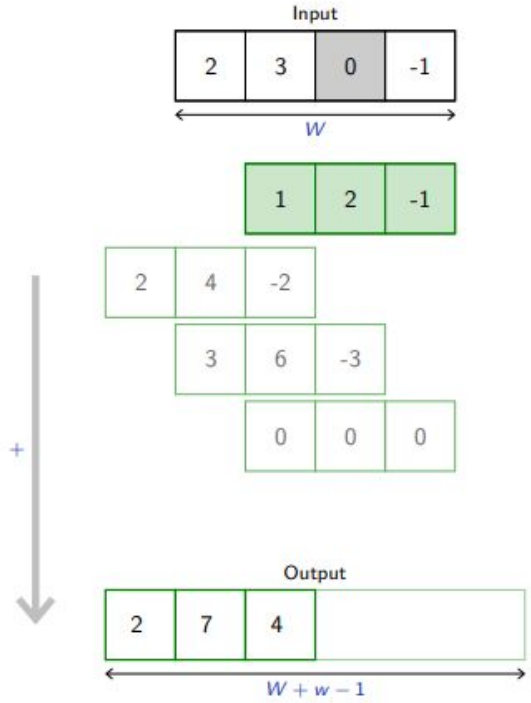


Lecture 5: Mathematically one can rewrite the convolution operation similar to MLP where the effective weight matrix is no longer a dense matrix.

$$\mathbb{R}^D \to \mathbb{R}^{D'}; \quad x \mapsto \sigma(wx + b)$$

$$\begin{bmatrix} 1 & 2 & 0 & -1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2 & 0 & -1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 2 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 2 & 0 & -1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 2 & 0 & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 2 & 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 2 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 2 & 0 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ 4 \\ -1 \\ 0 \\ 2 \\ -2 \\ 1 \\ 3 \\ 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 9 \\ 0 \\ 1 \\ 3 \\ -5 \\ -3 \\ 6 \end{bmatrix}$$

TansposeConvolution



Remember $W_g = W_f^T$

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ -1 & 2 & 1 & 0 \\ 0 & -1 & 2 & 1 \\ 0 & 0 & -1 & 2 \\ 0 & 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} 2 \\ 3 \\ 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 2 \\ 7 \\ 4 \\ -4 \\ -2 \\ 1 \end{bmatrix}$$

```
torch.nn.ConvTranspose2d(in_channels, out_channels,
kernel_size, stride=1, padding=0, output_padding=0,
groups=1, bias=True, dilation=1, padding_mode='zeros',
device=None, dtype=None)
```

```
F.conv_transpose2d(input, weight, bias=None, stride=1,
padding=0, output_padding=0, groups=1, dilation=1)
```

Convolutional AE: Example

```
AutoEncoder (
(encoder): Sequential (
(0): Conv2d(1, 32, kernel_size=(5, 5), stride=(1, 1))
(1): ReLU (inplace)
(2): Conv2d(32, 32, kernel_size=(5, 5), stride=(1, 1))
(3): ReLU (inplace)
(4): Conv2d(32, 32, kernel_size=(4, 4), stride=(2, 2))
(5): ReLU (inplace)
(6): Conv2d(32, 32, kernel_size=(3, 3), stride=(2, 2))
(7): ReLU (inplace)
(8): Conv2d(32, 8, kernel_size=(4, 4), stride=(1, 1))
)
(decoder): Sequential (
(0): ConvTranspose2d(8, 32, kernel_size=(4, 4), stride=(1, 1))
(1): ReLU (inplace)
(2): ConvTranspose2d(32, 32, kernel_size=(3, 3), stride=(2, 2))
(3): ReLU (inplace)
(4): ConvTranspose2d(32, 32, kernel_size=(4, 4), stride=(2, 2))
(5): ReLU (inplace)
(6): ConvTranspose2d(32, 32, kernel_size=(5, 5), stride=(1, 1))
(7): ReLU (inplace)
(8): ConvTranspose2d(32, 1, kernel_size=(5, 5), stride=(1, 1))
)
)
```

Convolutional AE: Example

1×28×28

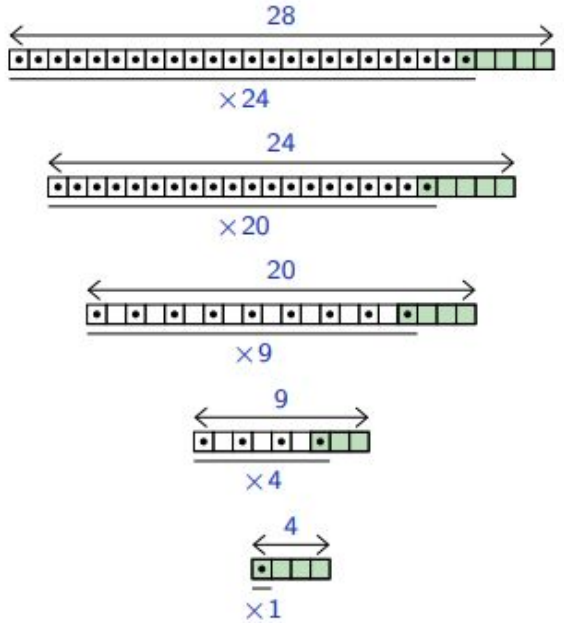
```
nn.Conv2d(1, 32, kernel_size=5, stride=1)
```
32×24×24

```
nn.Conv2d(32, 32, kernel_size=5, stride=1)
```
32×20×20

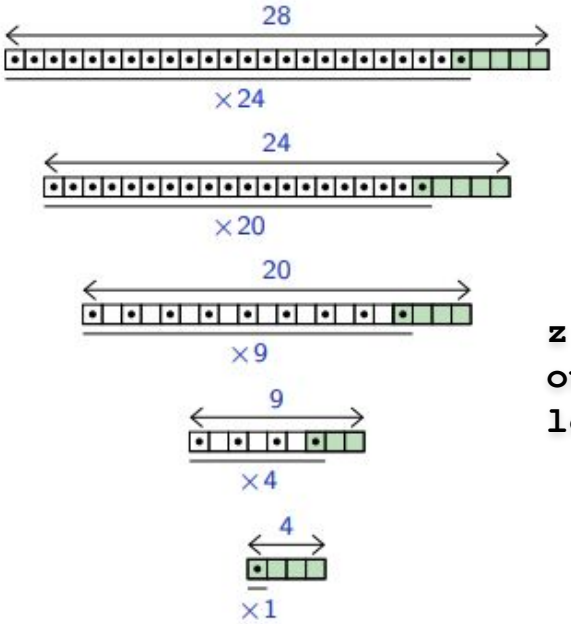
```
nn.Conv2d(32, 32, kernel_size=4, stride=2)
```
32×9×9

```
nn.Conv2d(32, 32, kernel_size=3, stride=2)
```
32×4×4

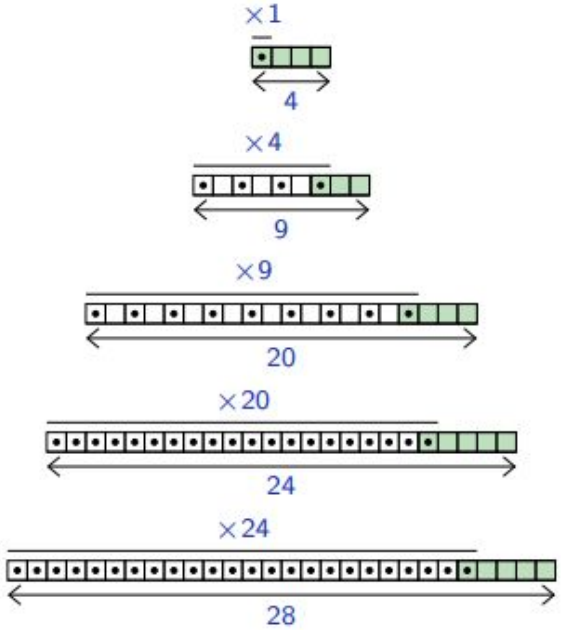
```
nn.Conv2d(32, 8, kernel_size=4, stride=1)
```
8×1×1



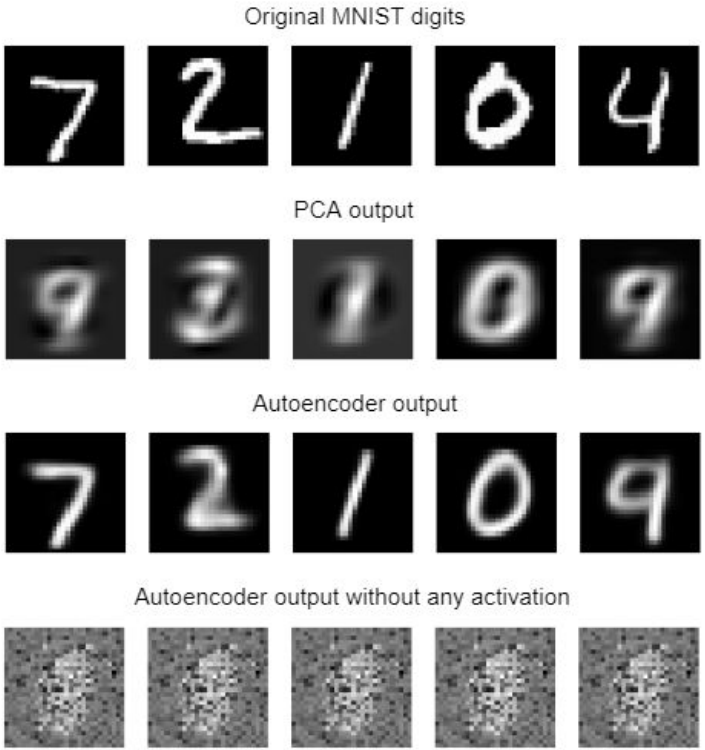
Convolutional AE: Example

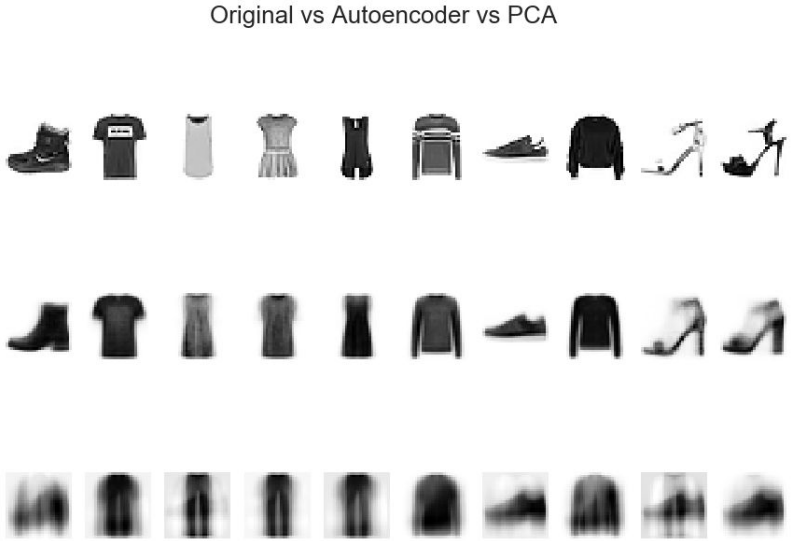



```
z = model.encode(input)
out = model.decode(z)
loss = MSE
```



Convolutional AE: Example

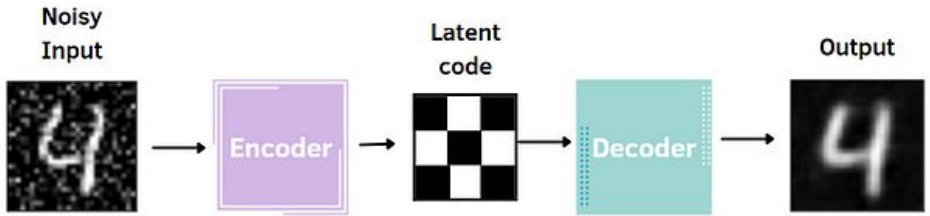


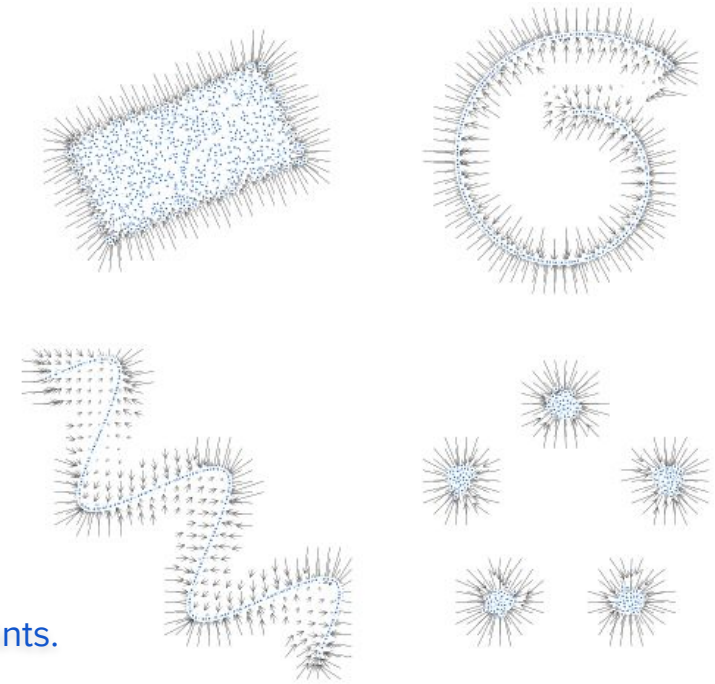


Denoising AutoEncoders (AE)

$$w_f, w_g = \arg\min \frac{1}{N} \sum_{n=1}^{N} \|x_n - g(f(x_n, w_f), w_g)\|^2$$

$$\sum \|x_n - g(f(x_n + \epsilon_n, w_f), w_g)\|^2$$



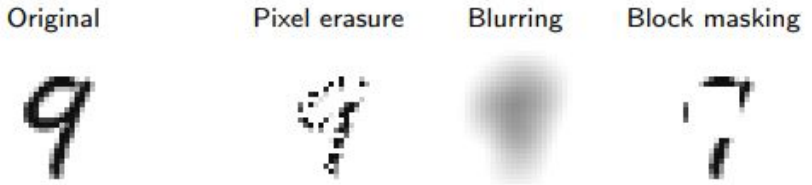


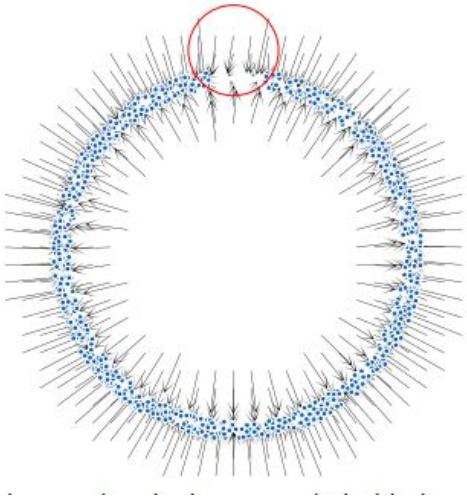
In each figure on the right blue dots are the training points.

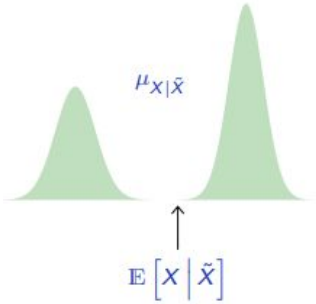
Arrows depict that the autoencoder learned to take the points “back” to the [support of the] distribution.

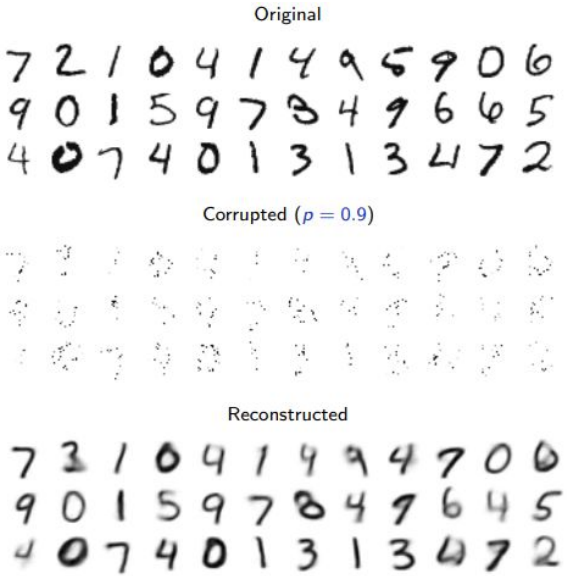
Denoising AutoEncoders (AE)

$$\sum \|x_n - g(f(x_n + \epsilon_n, w_f), w_g)\|^2$$









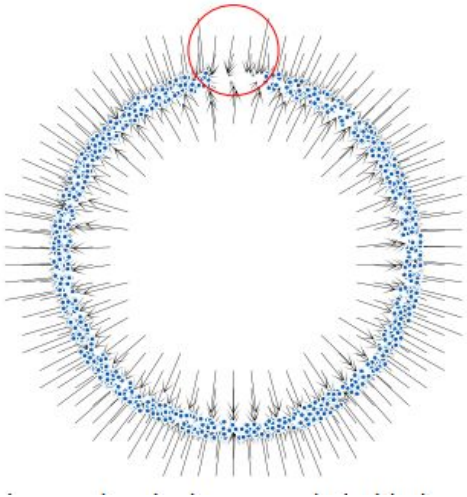
The output distribution can be multimodal due to which results are often not great.

Denoising Auto

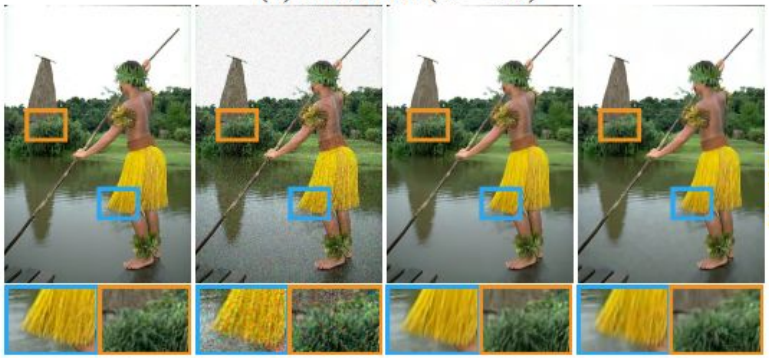
$$\sum \|x_n - g(f(x_n$$

Original

Pixel erasure



(a) Gaussian ($\sigma = 25$)

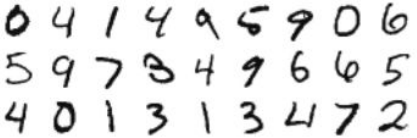


(b) Poisson ($\lambda = 30$)

(c) Bernoulli ($p = 0.5$)

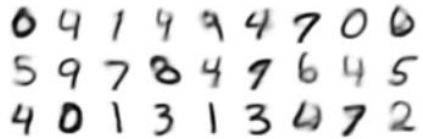
Ground truth | Input | Our | Comparison

Original



Corrupted ($p = 0.9$)

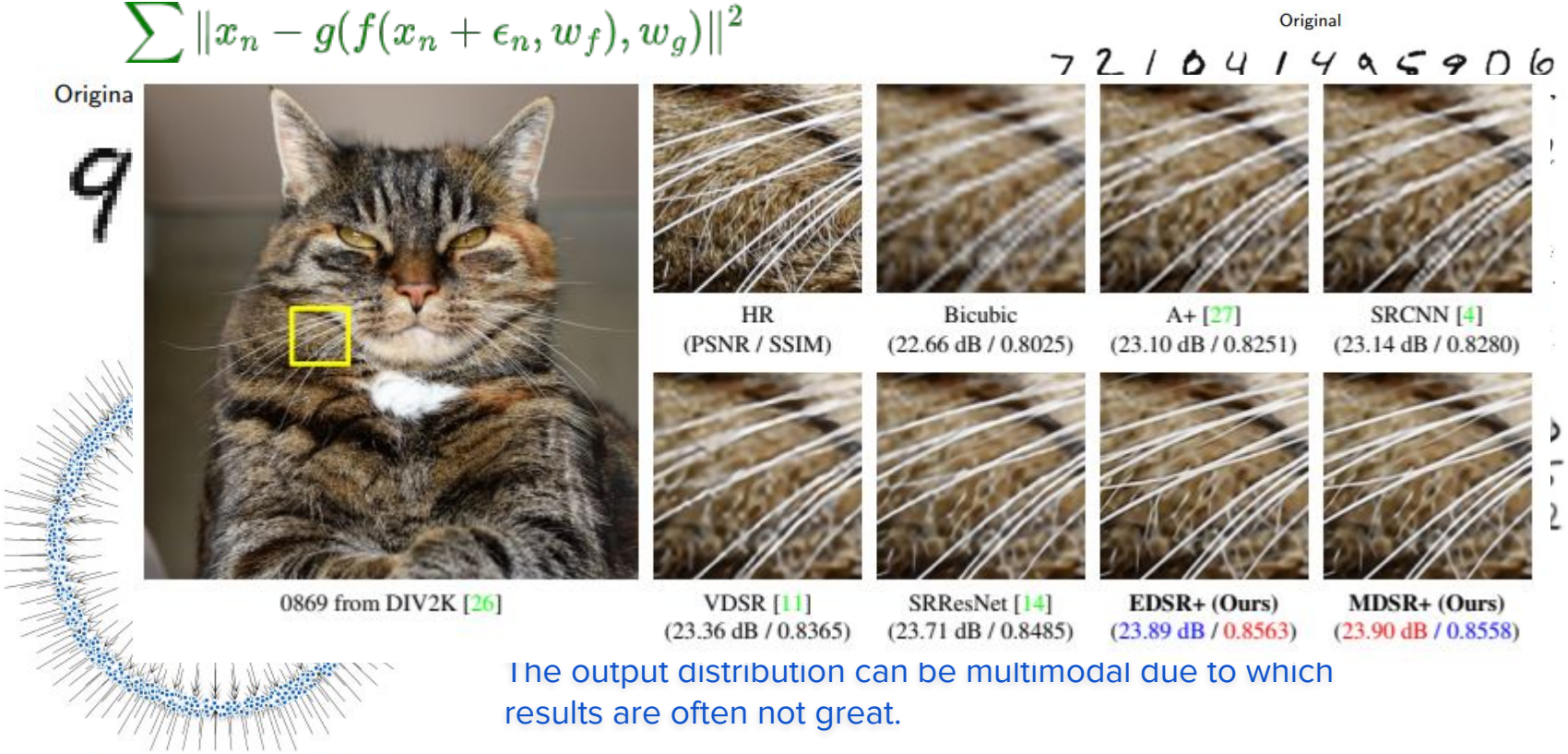
Reconstructed



ue to which

Denoising AutoEncoders (AE)

$$\sum \|x_n - g(f(x_n + \epsilon_n, w_f), w_g)\|^2$$



The output distribution can be multimodal due to which results are often not great.

Denoising AutoEncoders (AE): Noise2Noise Model

Consider noisy measurements **(y_1, y_2, y_3....)**

$$\arg\min_z \mathbb{E}_y L(z, y)$$

Estimating the true unknown input requires us to find a number **z** that has the smallest average deviation from the measurements.

For L2 loss: z=mean
For L1 loss: z=median

Example: Low-light photography - a long, noise-free exposure is the average of short of independent, noisy exposures.

Consider ϵ and δ two independent noises

$$\begin{aligned}
&\mathbb{E}[\|\phi(X+\epsilon)-(X+\delta)\|^2] \\
=&\mathbb{E}[\|\phi(X+\epsilon)-(X)\|^2]+\mathbb{E}[\|\delta\|^2] \\
&-2\mathbb{E}[\delta]\mathbb{E}[\phi(X+\epsilon)-(X)] \\
=&\mathbb{E}[\|\phi(X+\epsilon)-(X)\|^2]+\mathbb{E}[\|\delta\|^2]
\end{aligned}$$

$$\begin{aligned}
&\arg\min_\theta \mathbb{E}[\|\phi(X+\epsilon,\theta)-(X+\delta)\|^2] \\
=&\arg\min_\theta \mathbb{E}[\|\phi(X+\epsilon)-(X)\|^2]
\end{aligned}$$

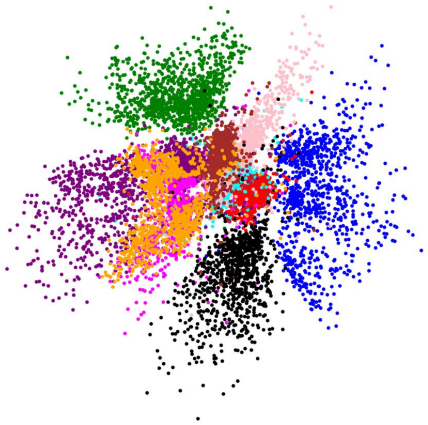
https://arxiv.org/pdf/1803.04189.pdf

AutoEncoders: Discriminative Ability

Learned Supervision in Latent Space via Unsupervised Training

- An AE is trained in an unsupervised manner. If we just consider the output of Encoder, we can say that it is a compressed compact representation/code of input
  - This code has to capture all invariant properties or variations from dataset
  - Otherwise reconstruction quality will be poor
  - What if AE reserves a few dimensions for one kind variation?
    - In simplest form this variation could be at class level!

In the image is shown the 2D projections of the latent vectors of an AE trained on MNIST.



Observe that there are approx 10 clusters. Labels?

There is an inherent clustering behaviour in latent space of most AEs
Because of which AEs are often used as feature extractors trained on unlabeled data.

Variational AutoEncoders (VAEs)

VAEs extend the deterministic latent space to probabilistic space by express their latent attributes as a probability distribution, leading to the formation of a continuous latent space that can be easily sampled and interpolated.